

Übungsblatt 4

Vorlesung „Secure Software Systems“ Wintersemester 2025/26

Praktische Übung Randomization

Arbitrary Read

Das Programm `ar` ist anfällig für einen Memory Corruption Angriff über eine Buffer Overflow Schwachstelle mit der ein Pointer überschrieben werden kann. Nutzen Sie die Schwachstelle in dem Programm aus, um ein *Arbitrary Read* Exploit-Primitiv zu konstruieren. Das Ziel ist es, damit den Inhalt des Strings `flag` auszulesen.

Aufgabe 1.

- i) Wo befindet sich die Buffer Overflow Schwachstelle in diesem Programm?
Als Teil eines *Arbitrary Read* muss immer ein Pointer manipuliert werden.
 - Welchen Pointer können Sie überschreiben?
 - Wo und wie wird dieser Pointer später verwendet?
 - Wie erhalten Sie dadurch einen *Arbitrary Read*?
 - Wie viele Daten können Sie dann damit lesen?
- ii) Erweitern Sie das Exploit-Template zu einem funktionierenden Exploit. Überschreiben Sie den Pointer so, dass das Programm den Inhalt des Strings `flag` ausgibt.

Bypassing Full ASLR

In dieser Aufgabe begegnet uns erneut eine Variante eines bekannten Programms: `getreal3`, welches wir in vorhergehenden Übungen bereits analysiert hatten. Das Programm ist funktional quasi identisch zu den vorhergehenden Versionen von `getreal` und kann noch immer mit einem Stack-basierten Buffer Overflow angegriffen werden. Ziel des Angriffs ist es wieder den Inhalt der Datei `secret.log` auszugeben. Im Unterschied zu vorhergehenden Übungen ist diesmal jedoch ASLR vollständig aktiv, es also gibt keine fixen Adressen im Adressraum. Dadurch müssen Sie zuerst einen Weg finden, die aktuellen Adressen von nützlichen Gadgets zur Laufzeit wiederzufinden (Information Leak).

Aufgabe 2.

- i) Finden Sie eine Möglichkeit ASLR zu umgehen. Identifizieren Sie dazu einen Information Leak Bug, mit dem es möglich ist, Adressen auszulesen. Erklären Sie den Bug und wie Sie ihn dazu nutzen können, um die Adressen der Gadgets zu berechnen.
- ii) Identifizieren nützliche ROP-Gadgets z.B. in dem Programm oder der `libc`. Erklären Sie welche ROP-Gadgets Sie verwendet haben.
- iii) Erweitern Sie `exploit.py` zu einem funktionierenden Angriff. Ihr Angriff soll mindestens die erste Zeile der Datei `secret.log` auslesen und ausgeben.

Bypassing Stack Canaries

Stack Canaries (oder Stack Cookies) werden verwendet, um Buffer Overflow Angriffe zu erkennen. Wurde ein Angriff erkannt, terminiert das Programm anstatt zur Rücksprungadresse zu springen. Unter Linux kann man diesen Schutzmechanismus daran erkennen, dass geschützte Funktionen einen Sprung zur Funktion `__stack_chk_fail` enthalten, welche das Programm terminiert, wenn das Canary nicht mehr intakt ist. Das 64-Bit x86 Programm `3_cookie/cookie` enthält einen stackbasierten Buffer Overflow. Das Programm wurde jedoch mit `-fstack-protector-strong` kompiliert, was die Stack Canary Mitigation aktiviert. Allerdings kann das Programm trotzdem erfolgreich angegriffen werden. Diese Übung soll verdeutlichen, dass Schutzmechanismen auch umgangen werden können.

Aufgabe 3.

Erweitern Sie das Exploit-Template zu einem funktionierenden Angriff, der zu der Funktion `shelly` zurückspringt. Erklären Sie, wie Sie den Stack-Canary-Schutzmechanismus umgangen haben.

Just-In-Time Return-Oriented-Programming

Als Angreifer haben Sie nicht immer Zugriff auf die binäre Programmdatei. Wenn Sie beispielsweise einen Web-Server angreifen, können Sie meistens feststellen, um welchen Web-Server es sich handelt und ggf. sogar die genaue Softwareversion bestimmen. Da viele Web-Server open-source sind, können Sie nun den Code nach Schwachstellen durchsuchen, die einen Angriff ermöglichen. Allerdings kennen Sie dadurch nicht die exakte Binary, die auf dem Server läuft, die abhängig vom benutzten Compiler und Betriebssystem ist. Dies betrifft insbesondere die genauen Adressen und Gadgets, die Sie für einen Exploit benötigen. Diese Aufgabe soll verdeutlichen, dass auch unbekannte Programme mittels ROP angegriffen werden können, wenn ein Information Leak für einige Code-Regionen vorhanden ist. Wir verwenden hier also eine etwas vereinfachte Variante der *Just-In-Time Return-Oriented-Programming* (JIT-ROP) und suchen Gadgets zur Laufzeit des Programms ohne dieses vorher zu kennen.

Das Programm `random` simuliert einen Information Leak eines unbekannten Programms, indem es *zufällig* Code-Bytes generiert und ausgibt. Weiterhin sind Schutzmechanismen wie ASLR aktiviert, sodass die Adressen von Funktionen nicht konstant sind.

Aufgabe 4. (optional)

- i) Schreiben Sie einen Exploit, der in den ausgegebenen Code-Bytes nach sinnvollen ROP-Gadgets sucht und eine Chain generiert, die die `secret_function` mit korrekten Argumenten aufruft.

- In welchem Adressbereich befindet sich die gegebene Code-Region?
- Wie finden Sie die Adresse der Zielfunktion? Wie rufen Sie diese auf?

Da diese Aufgabe auf zufällig generierten Daten basiert, ist es möglich, dass nicht immer genügend Gadgets vorhanden sind. Diese Aufgabe sollte allerdings in etwa 99% der Ausführungen lösbar sein. (Ein Exploit mit höherer Fehlerrate ist zur Lösung ausreichend.)

- ii) Erstellen Sie einen Exploit für das Programm `random_hard`, das sich nur in einer Zeile von dem vorherigen unterscheidet, nach denselben Vorgaben.

- Wie finden Sie die Adresse der Zielfunktion? Analysieren Sie dazu den Programmzustand direkt vor dem Aufruf der ROP-Chain.

Diese Aufgabe erfordert etwas komplexere Gadgets, sollte aber noch in etwa 28% der Ausführungen lösbar sein. Suchen Sie nach möglichst kurzen Gadgets, bestehend aus wenigen Bytes, um diese mit höherer Wahrscheinlichkeit in der gegebenen Region wiederzufinden.